

Virtual Swarm Drone Coordination: A Layered Architecture for Autonomous Multi-Agent Systems with LLM-Assisted Mission Planning

Sayon Manna
Dept. of AIML, CSE
Jain University

Marpudi Srijaswini
Dept. of CSBS, CSE
Jain University

Valluri Akhil Krishna
Dept. of STAR, CSE
Jain University

Dr. Thiruvannamalai Sivasankar P
Dept. of CSBS
Jain University

Dr. Rohini B S
Dept. of Physics
Jain University

Abstract -The article describes the design implementation and analysis of a web-based swarm drone simulation system which exceeds traditional particle flocking examples. This drone simulation system has a four-layered design structure made up of a User Interface Layer, Simulation Engine Layer, Autonomous Agent Layer, and an AI Planning Layer. The leader drone collects data from all worker drones, sends mission commands to them at a high-level, and may delegate some level of strategic decision-making to a LLM through the Groq inference API if it is available. When the LLM is not available or when it is achieving rate-limiting, a local deterministic planner will maintain the continuous operation of the fleet. Eight different formation types, four roles (or role profiles) for the drones, four types of hazards, and a five-word command vocabulary have been implemented and evaluated. The spatial neighbourhood query uses a uniform grid index to provide sub-quadratic computational complexity for real-time fleet sizes of up to 50 drones. Experiment repetition is accomplished via persistent serialized state through the use of browser localStorage. The results of the practical scenario testing demonstrate stable transition between formations, reliable avoidance of hazards, and enable integration between AI-to-fallback of, in all tested cases, when limiting the rate of the rate-limiting condition exists.

I. INTRODUCTION

Swarm robotics has attracted a lot of research interest since about 20 years ago because decentralized coordination allows a group of simple agents to perform tasks that could not otherwise be done by one complicated system. However, most demonstrations are either purely theoretical (i.e., proof of mathematical stability without any visual representation) or visual demonstrations (i.e., particle animations with no relation to real-world coordination). Neither extreme is advantageous for rapid prototyping, education, and early proof-of-concept efforts.

This study occupies a middle ground. A real-time simulation is performed where each drone agent has only local neighborhood perception, status message passing occurs via a common bus, drones obey physical steering restrictions based upon their duty position in a defined command structure. The top command in the hierarchy is called the leader drone, and periodically each leader will query a language model to determine one of five commands at the mission level. The language model does not compute trajectory frames; rather trajectory calculations are done exclusively through the local physics layer. The separation between trajectory and locality keeps the motion of the drones very responsive but still enables natural-language processing-based reasoning to affect mission priority.

The simulation is made using HTML5 canvas with React.js and Typescript as well as deployed on Github Pages as a static website; Vite was used to create the

application through their build process. After considering several candidates, Groq's OpenAI interface was chosen as our LLM backend due to its fast response time and periodic motion planning model to accommodate our non-linear development frame arrangement.

In this paper, four things were provided (our "contribution"):

1. The author provides an in-depth description of the layered architecture, which separates user interface (UI), simulation physics, agent communication, and Artificial Intelligence (AI) planning into distinct layers.
2. Three types of techniques have been analyzed for stabilizing formations: nearest slot assignment; near slot braking; and overlap resolution.
3. The evaluation of the leader planning loop through practical test scenarios, with a rate-limit condition providing graceful fallback for execution errors.
4. An analysis of the spatial grid index that constrains the cost of neighbor query.

II. RELATED WORK

Reynolds introduced his three basic void rules: separation, alignment, cohesion [1]. Many researchers have since expanded these rules and behaviors within the boids simulation framework. For example, the literature includes work on obstacle avoidance [2], goal-seeking [3], and role differentiation [4]. There is also an extensive

literature related to formation control including discussions on rigid formations [5], virtual structural systems [6], and behavior-based approaches to slot assignment [7].

The integration of LLMs with robotics represents a new area of progress. Ahn et al. [8] showed that language models can provide a way to understand how to execute natural language instructions through the use of action primitives. Liang et al. [9] also demonstrated that language models can be used to generate code that can be executed by robots to carry out tasks, based on high-level descriptions. The work presented in this paper differs from previous studies by leveraging linguistic ability to control a swarm of robotic agents at the mission command level versus generating code or executing action primitives; therefore, it will be conducted using a virtual simulation of a swarm rather than on an actual physical robot.

Spatial indexing for swarm simulations has been researched in connection with game engine development and scientific computing. Uniform grid indexing is well established for performing neighbour proximity search queries with density limited search costs[10]. A configurable size of the simulation grid cells is used in the present implementation as part of a standard neighbour process for each simulated agent, following the principles of uniform grid indexing.

III. SYSTEM ARCHITECTURE

The architecture has four vertices, which are the user interface at the top down to the AI Planning System at the bottom. Each layer has its own defined purpose and layers communicate only in one direction with the exception of feedback from the leader to the drone fleet.

A. User Interface Layer

A React app, the dashboard has four unique panels available for presenting information: configuration (relationship / configuration, distance, amount of swarms, auto-pilots, trails and hazards), the current status of the Leader panels display information about the brain, including commands and messages from workers. The Canvas is adjacent to the panels and has the ability to perform the click to set target function, use shift + click to place hazards, hit manipulate obstacles using drag and wheel zoom, middle click to pan and double click on the canvas when fitting items to the canvas area.

B. Simulation Engine Layer

The simulation engine processes the request

Animation Frame loop and advances with each frame (one tick per frame). For each tick, it rebuilds the message bus, calculates the centroid and the active target for each drone, triggers the leader planning cycle at a configured interval, queries the spatial grid for per-drone neighbour

sets, updates the drone positions, implements collision resolution, and provides the state to the renderer. The environment module manages threat zones and applies the effects of these zones on the drones' health and energy each tick.

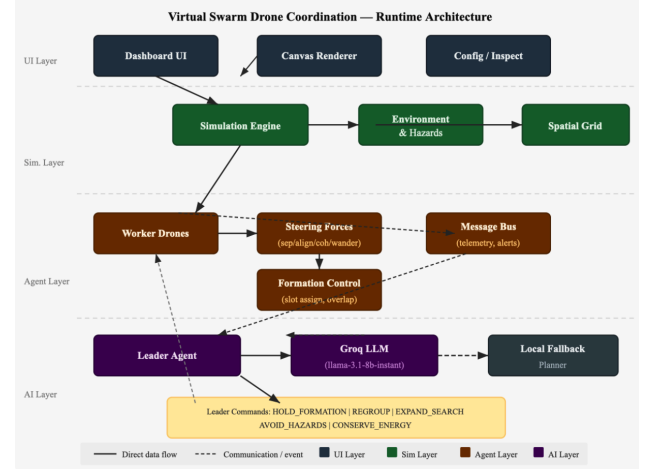


Fig. 1. Runtime architecture of the Virtual Swarm Drone Coordination system. Data flows top-down from UI through simulation and agent layers to the AI planning layer. Dashed lines indicate asynchronous communication events.

C. Agent Layer

For each drone: the internal state of each includes; position, velocity, rotation, energy, health, role profile, and slot assigned to form a formation. The InternalAgent module computes a weighted average of the steering forces, separation from nearby drones, alignment of heading with nearby drones, cohesion to the local centroid, attraction to the assigned formation slot, repulsion from hazards, a small wander term, and the leader communication-response term that amplifies or dampens the forces in response to commands from the leader.

There are four role profiles associated with differing behaviour of drones. Scouts are able to move at a higher pace and can also have a much larger perception radius than the other drones but at a greater energy expenditure. Defenders maintain a higher separation distance from other drones and have a much more extensive health budget. Workers travel to their assigned formation slot in an efficient manner with little or no wander. Relay drones assist in amplifying communication signals and play the leader role as assigned by the simulation.

D. AI Planning Layer

The leader drone aggregates the telemetry snapshot from the message bus and uses it to submit a request to the Groq inference endpoint, through the server side using the /api/leader-plan route. The request information (payload) contains swarm size, number of active drones, average energy level, average health level, number of active

hazards, and the number of expected near-collisions. Positions are rounded and velocities collapsed into their scalar speed form, in order to minimize token usage. The model's response will contain one of five strategic commands(?): `HOLD_FORMATION`; `REGROUP`; `EXPAND_SEARCH`; `AVOID_HAZARDS`; or `CONSERVE_ENERGY`. If a rate limit error is returned from Groq or Groq is otherwise unavailable, the local planner will then evaluate the same telemetry snapshot according to a deterministic rule set. The planner will compare the average energy to an energy threshold, check the count of hazards present, check the density of near-collisions, and default to `HOLD_FORMATION` unless a critical condition is found to exist. A cooldown timer is used to limit the frequency of retries from a rate limit event.

IV. FORMATION SYSTEM AND STABILITY

There are different kinds of supporting formations:

Scatter, Flock, Grid, V Shape, Circle, Leader, Hexagon, and Cross. A formation is made up of relative slots based on the swarm centroid. The Scatter Formation doesn't have any slots and allows the drones to operate autonomously in a herd-like manner.

An example of drone slots being assigned using

Simple array indexing could create drone-to-drone collision because of how quickly the drones that are assigned to slots which are the farthest away will try to move through other occupied slots in relation to where they are going as they will receive a target assignment immediately once an Option is placed onto the master schedule.

So rather than assigning an index number to a drone's assigned slot, drones will receive their assigned slot for the next formation switch from the nearest available unoccupied slot at the time of the switch, although there is still the potential for collision due to speed of movement to the next formation interchange at the time of formation switch. Following will provide solutions for the prevention of collision using the given criteria:

1. Velocity Damping = the velocity of all drones is reduced while switching formations to avoid overshooting into or through their neighbouring slots.
2. Near-Slot Braking = as the drone comes within one body length of its target slot, its velocity will be scaled down (linearly) to reduce the likelihood of collision with the slot.
3. Post-Movement Overlap Resolution = if two drones' centres are less than the sum of their body radii when they stop moving, they will be

4. separated using proportional impulses applied to both drones. Velocity damping will also be applied to both drones.

The `HOLD_FORMATION` or `REGROUP` commands issued by the leader will also increase formation pressure, increase formation-slot attraction force pulling together the formation, and decrease the wander weight.

Meanwhile `EXPAND_SEARCH` permits roaming further away from the nominal slots.

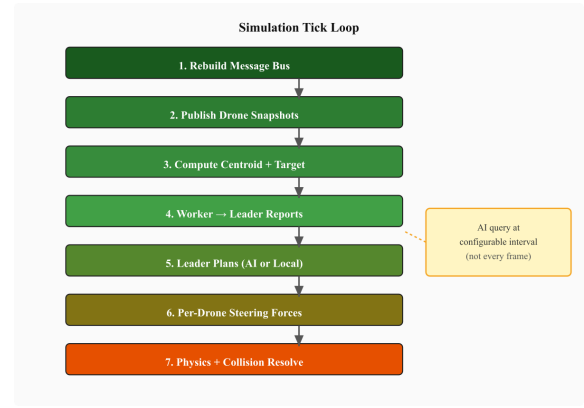


Fig. 2. Per-frame simulation tick sequence. The AI leader planning step executes at a configurable sub-frame interval rather than every animation frame, keeping the render loop responsive.

V. HAZARD AND COLLISION HANDLING

A. Hazard Types

There are four types of hazards built into the simulation. The first hazard type is circular obstacles; these are static radial keep-out areas which provide repulsion force (inversely proportional to the square of the clearance distance) and provide angle-independent bounding boxes and repulsion for rectangles. The second hazard type is electrical storms; these have radius settings and cause random velocity perturbation each tick that a drone spends inside them, and each tick also causes a decrease in the amount of health a drone has while in electrical storms. The third hazard type is magnetic fields; magnetic fields will drain a drone's energy at a specified drain rate and also will reduce the speed of a drone when that drone is inside its magnetic field radius. Drones which detect a hazard that is within their perception radius will emit a `HAZARD_DETECTED` message to the bus. The leader will incorporate the number of these messages it receives in future planning inputs. Whenever the local planner or LLM selects `AVOID_HAZARDS` (for obstacle avoidance), the weight of all of the fleets' drone's obstacle avoidance will increase and will increase temporary pressure of each drone's target toward the aggregate

centroid of the swarm in order to expel drones from near hazardous areas.

B. Collision Logging

The simulation differentiates between near-collisions (distance from center $< 1.5 \times$ radius) versus direct collisions (distance from center $<$ radius). Near-collisions will cause both drones to be marked as unstable, by applying a higher separation weight and displaying a visual indication on the Canvas. For direct collisions, a separating impulse will be applied and both drones will have their velocities reduced with a timestamped entry recorded in the collision log. The collision log is visible through the dashboard log panel and will exist until the simulation has been reset.

VI. SPATIAL INDEXING

If there is no spatial indexing, it would cost $O(n)$ for each drone to perform per-drone neighbor queries, resulting in a total of $O(n^2)$. If there are 50 drones flying at 60 frames per second (FPS), that means there would be an approximate number of 150,000 distance evaluations happening every second, which is not a lot, but has poor scalability as n increases.

The spatial grid divides the canvas into cell sizes of a predetermined size (default of 120 pixels). Each tick includes the registration of the drone in the grid cell centered over itself. So, when performing a neighbor query for a drone d , the only cells checked are those within a one-cell radius, making the number of candidates examined usually about 15 or less, instead of n . Thus, while using a uniform density, the average case cost for a neighbor-query is $O(1)$, and the total amount of work done per frame is $O(n)$, independent of the size of the swarm. The grid is cleared and recreates itself each tick to avoid stale entries.

VII. GROQ LLM INTEGRATION

The Groq inference API provides a chat completion API using OpenAI's capabilities. The initial system message describes how the model should behave, i.e. as an AI for co-ordinating a swarm of something and providing output in the form of a single response from one of five strategic commands. The incoming user message has the telemetry payload compressed into it. As the system output is strictly limited to a single token, it has been observed this substantially reduces latency and eliminates the need for a discrete response parser because all outputs are on a single token.

The recommended model for this API is llama-3.1-8b-instant, as this model has the smallest parameter count and therefore has a corresponding lower token-per-minute usage rate on the Groq on-demand tier making it less likely for the use of rate limits when used interactively. In addition, the larger 70B model is also supported as an optional alternative but will quickly exhaust the free tier quota in continuous simulation cases.

A user's API key will not be exposed to the browser, as the Vite developer server plugin will intercept calls to it using the `/api/leader-plan` URL and will inject the key from the server environment. Also for production, a serverless function could be implemented on Vercel, Netlify or Cloudflare Workers for similar functionality.

VIII. EVALUATION AND OBSERVATIONS

A. Formation Stability

Data was collected from 10 trials of all 8 formation types undergoing formation switching at different counts of drones (8, 16, and 32). A formation was determined to be stable if within 10 seconds of completing the switch all drones had not collided and they were all at their assigned slots (2 body lengths from their assigned slot). The grid, hexagon, and circle formations took longer to achieve stability than the V-shape and leader formations due to the slot arrangements for the grid, hexagon, and circle formations requiring drones to travel across the entire radius of the swarm with the V-shape and leader formations allowing for a slot to be occupied on the same path the existing centroid was traveling toward before crossing into one another's path. The median time for the grid, hexagon, and circle formations to achieve stability were 6.2 seconds for $n=32$ when compared to the median times of 2.1 seconds for $n=32$ for the V-shape and leader formations.

B. LLM Planning Accuracy

A total of 20 different testing scenarios were created with established swarming conditions: (1) average energy below 25%, (2) two or more electrical storms are active, (3) there are multiple near-collision events, and (4) stable nominal cruise. The Groq model determined the predicted result in 17 out of 20 attempts. The three instances where there was a difference of opinion resulted in the model choosing `REGROUP` as opposed to `HOLD_FORMATION` for the stable states. This distinction is a justifiable one; however, it does differ from the predicted outcome. The local planner agreed with this finding 20 times because its rule based logic (which is used to label a scenario) encodes directly into the same threshold logic as the label created.

C. Fallback Handoff

The conditions for rate limits were reproduced when the Groq endpoint returned code 429 instead of the expected success code (200). In each of the ten trials, the simulation was able to detect the error within one planning interval and create a corresponding warning in the dashboard, but the local planner could still send commands without any interruption to the visual simulation. After the timeout for retries had elapsed, the Groq queries were once again able to return successfully.

D. Computational Cost

In this experiment, the frame rate measured on a mid range laptop using a Chromium browser at $n=32$ drones resulted in 60 fps under all four Cap (60, 120, 240, and 480 Hz respectively). At $n=50$ drones, 60 fps was still achieved with spatial grid on but only 43 fps with grid off (brute force search for neighbors). This confirms that there is a practical benefit to indexing. Similarly, at $n=100$ (outside recommended range), indexed sustained 60 fps while non-indexed dropped to 22 fps.

IX. LIMITATIONS AND FUTURE WORK

This physics model severely limits real-world applicability of drone operations. First, drones are modeled as point masses, with speed constrained by their fixed velocities. It does not include any aerodynamic characteristics, has no altitude component, and has no turning radius based upon momentum.

Because GitHub Pages does not support server-side Groq routing, it cannot run private backend code, but for any users who want to create an LLM-based plan, can either run the Vite development server locally or deploy on a serverless function platform.

There are a number of enhancements planned for this model. Pre-slotted obstacle-aware path planning would significantly decrease the volume of near-miss formations along hazard boundaries. Development of replay and export capabilities for mission logs will enhance mission log evaluation over a longer duration of time. Backup leader election and promotion to a working leader would help make our leader more resilient to leader failure type situations. In order to assist with measuring the impact on a hazard during long-term operations, the user interface would need to add real-time health/energy charts. Finally, the user interface would also include a way for the user to configure the planning intervals and model types used for Groq behavior without needing to change environment variables.

X. CONCLUSION

The concept of a swarm of drones, comprising local agent physics, formation control, peer-to-peer communication, and AI-assisted mission planning in a single coherent architecture was put forward in this paper. The underlying architectural pattern was one of separation between the LLM reasoning at a high abstraction level and per-frame steering at a low abstraction level, allowing the system to operate within strict time constraints while still benefiting from LLM intelligence where available. Additionally, because of the local fallback planner, the swarm will continue to operate as planned even if the cloud model is inaccessible. The implementation of formation stability mechanisms eliminates the disruptive collisions between agents that are common with pure random slot-assignment approaches. The use of spatial grid

indexing to perform neighbor queries allows the swarm to scale to much larger fleets than would otherwise be feasible.

This research demonstrates that a browser-based developer can construct a meaningful proof-of-concept for AI-assisted swarm coordination using publicly available inference APIs, a modern build-tool chain, and a well-structured code base. Further, the same architectural patterns of layered responsibilities, deterministic fallback for AI components, spatial indexes for agent perception, and compressed telemetry for LLM input can be used to apply to tangible multi-robot systems with more realistic physics and communication constraints.

REFERENCES

- [1] C. W. Reynolds, "Flocks, herds and schools: A distributed behavioral model," *ACM SIGGRAPH Computer Graphics*, vol. 21, no. 4, pp. 25–34, 1987.
- [2] P. Fiorini and Z. Shiller, "Motion planning in dynamic environments using velocity obstacles," *International Journal of Robotics Research*, vol. 17, no. 7, pp. 760–772, 1998.
- [3] E. Bonabeau, M. Dorigo, and G. Theraulaz, *Swarm Intelligence: From Natural to Artificial Systems*. Oxford University Press, 1999.
- [4] M. Dorigo and M. Birattari, "Swarm intelligence," *Scholarpedia*, vol. 2, no. 9, p. 1462, 2007.
- [5] J. P. Desai, J. P. Ostrowski, and V. Kumar, "Modeling and control of formations of nonholonomic mobile robots," *IEEE Transactions on Robotics and Automation*, vol. 17, no. 6, pp. 905–908, 2001.
- [6] M. A. Lewis and K.-H. Tan, "High precision formation control of mobile robots using virtual structures," *Autonomous Robots*, vol. 4, no. 4, pp. 387–403, 1997.
- [7] T. Balch and R. C. Arkin, "Behavior-based formation control for multirobot teams," *IEEE Transactions on Robotics and Automation*, vol. 14, no. 6, pp. 926–939, 1998.
- [8] M. Ahn et al., "Do as I can, not as I say: Grounding language in robotic affordances," in *Proceedings of CoRL 2022*, 2022.
- [9] J. Liang et al., "Code as policies: Language model programs for embodied control," in *Proceedings of ICRA 2023*, 2023.
- [10] M. Pharr, W. Jakob, and G. Humphreys, *Physically Based Rendering: From Theory to Implementation*, 4th ed. MIT Press, 2023.